

Final Project Report

Project Title

AI-Enabled Hybrid Book Recommendation System

Course: INFO 556 Information Retrieval and Web Search

Faculty: Dr Xiao Hu

Associate Professor

College of Information Science, UofA

Name: Avikumar Talaviya

SID: 23946784

Submission Date: 12/08/2025

Table of Contents

- 1. Introduction**
- 2. Objectives and Purpose**
- 3. System Functions**
- 4. Implementation Approach**
- 5. System Diagram**
- 6. Challenges**
- 7. Reflection**
- 8. References**
- 9. Appendix**

Introduction

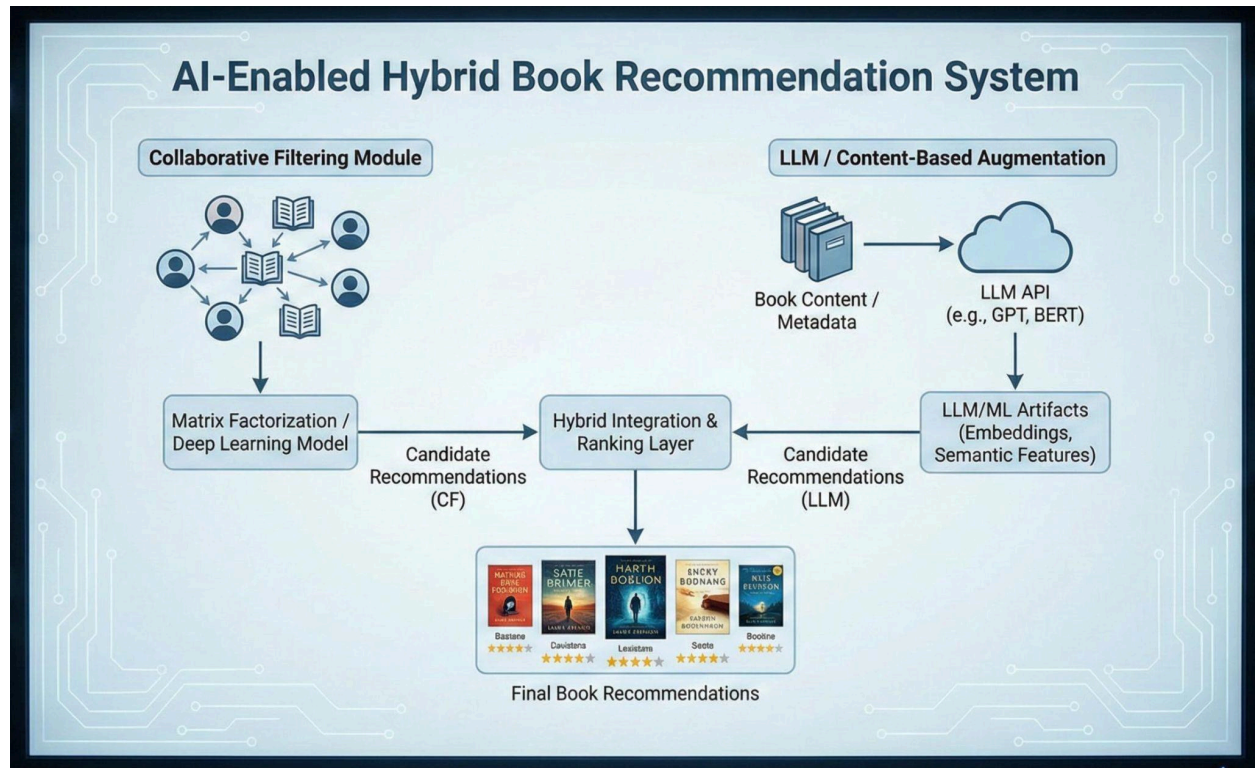


Figure 1: Introductory image of the hybrid recommendation image

Identifying the right books for academic or personal growth purposes can be a daunting task for many students and users across the school ecosystems, particularly those relying on traditional search engines, which often rely on keyword searches. These search engines might produce irrelevant results instead of relevant books that are in the interests of the students and the school community. These systems include the ISBN-based user search, which uses the generative AI model capabilities to expand the book titles to retrieve the relevant books. Hence, this approach can be widely helpful for book search tasks while keeping the search experience engaging and user-friendly.

To eliminate the redundant search results, this project introduces an AI-enabled recommender system that takes users' interests into account by expanding the query based on the search query using generative large language models and retrieves relevant books with authors and publication information from the databases. The system combines content-based filtering using item-user matrices, text clustering techniques and generative AI models to produce accurate and precise book recommendations to the users.

Objectives and Purpose

The primary goal of this project is to significantly enhance the book discovery process for students and general users by moving beyond simple keyword matching. The system utilised an ISBN-based search engine methodology to find the relevant book, leveraging a large language model with a reranking function. The system is designed to streamline the search experience, effectively reducing the time and effort required for users to locate relevant reading material. A central objective is to demonstrate the power of a hybrid approach, combining the collective intelligence derived from user behaviour in collaborative filtering with the contextual understanding of generative AI. By leveraging text clustering and AI-generated descriptions, the project aims to present recommendations in a user-centric manner that is both intuitive and informative. Furthermore, the system utilises large language models to perform semantic expansion on user queries, identifying relevant themes and titles that might otherwise be missed by standard search algorithms. This project also aims to demonstrate the user-centric design and feedback-driven approach to enhance the current recommendation systems.

System Functions

The system operates through a Streamlit-based user interface that specifically takes a book ISBN as input to initiate the recommendation process. This user query is then immediately fed into a backend API call that utilises a Large Language Model to generate relevant new book titles or broader themes that align semantically with the specific book identified by the ISBN. This query also retrieves similar book titles from the user item matrix database.

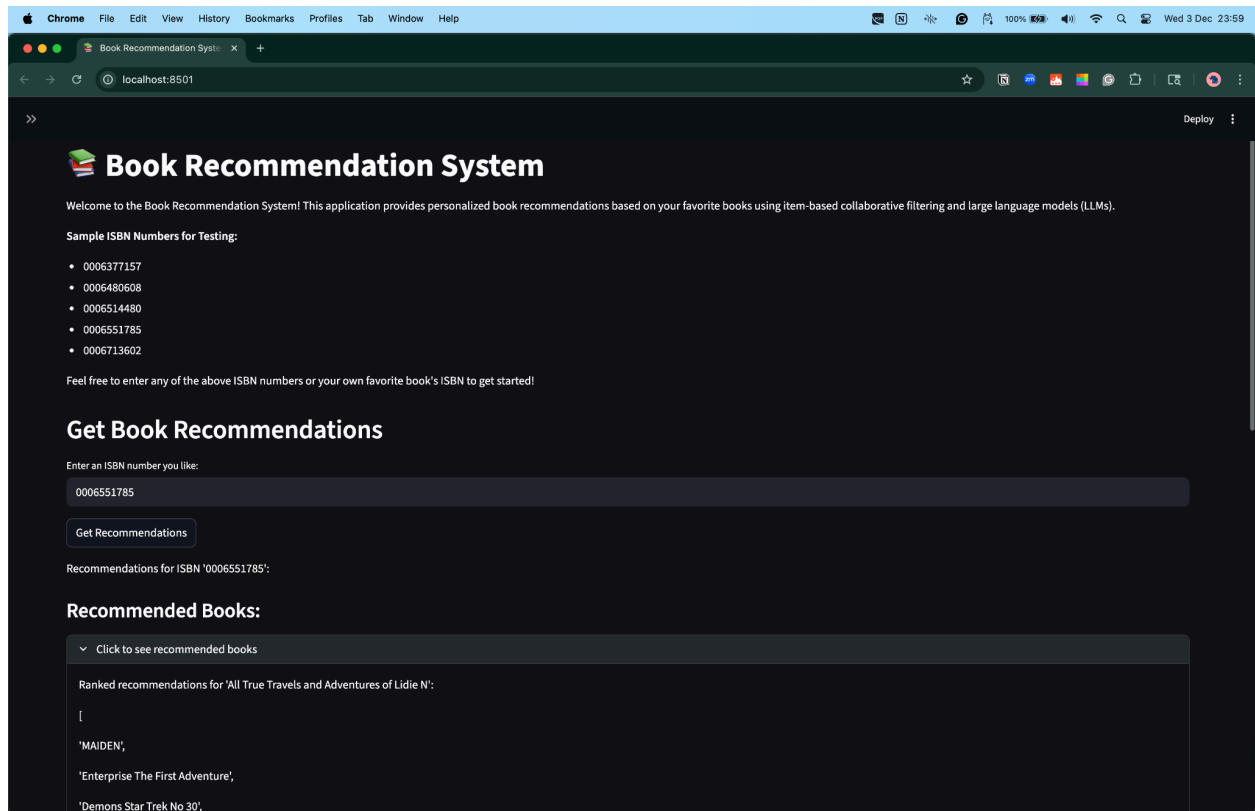


Image 1: Book recommendation system front demo page

In parallel with the semantic expansion, the system queries its internal database to find similar items using a specific item similarity algorithm, which then gets reranked based on the relevancy of the book titles using large language model API calls. The Generative AI-generated keywords are also used to retrieve relevant book titles from the database, effectively finding similar items to the user query, which are then reranked based on relevance to ensure high-quality suggestions.

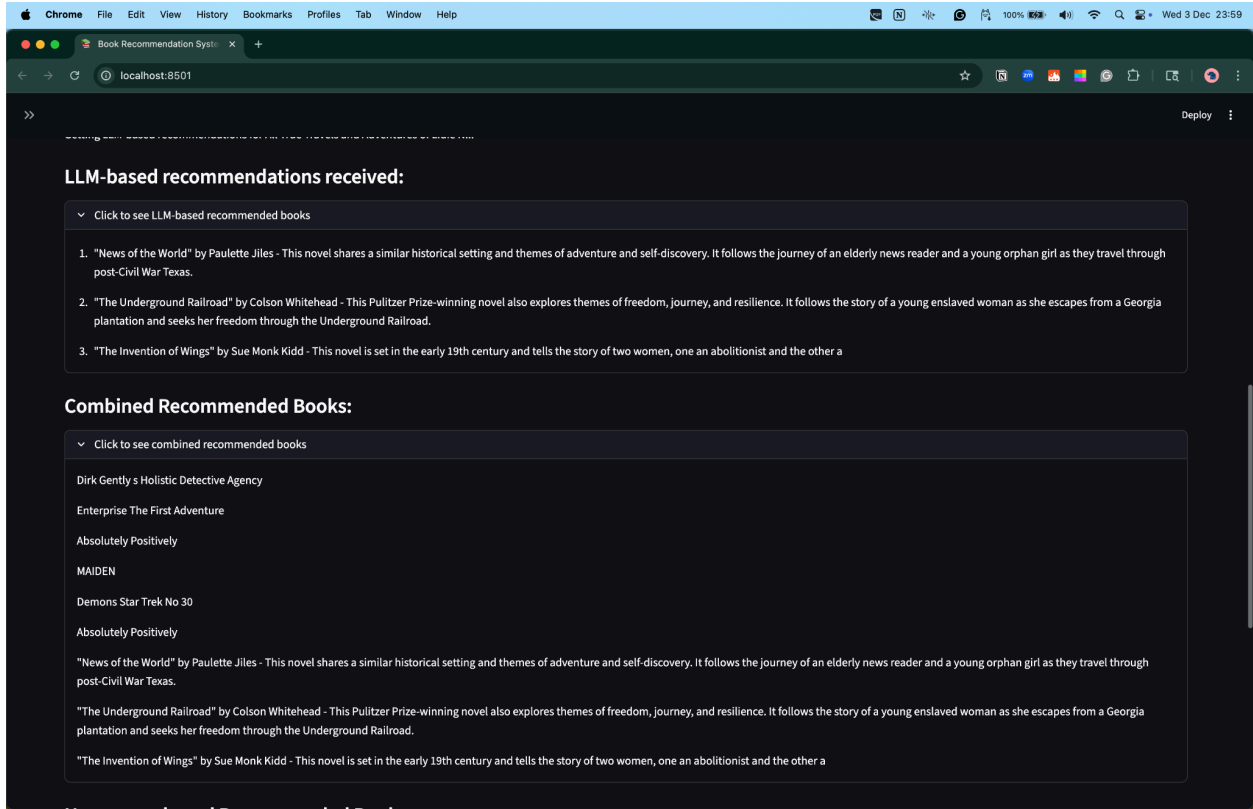


Image 2: LLM-based and combined recommendation front demo page

The combined results from both retrieval methods are subsequently clustered into groups using the K-means algorithm, which allows the system to generate detailed, context-aware descriptions for each recommendation using a secondary Large Language Model API call. These comprehensive results are finally presented to the user on the interface, where a feedback loop is incorporated to allow users to provide input on the recommendations they receive.

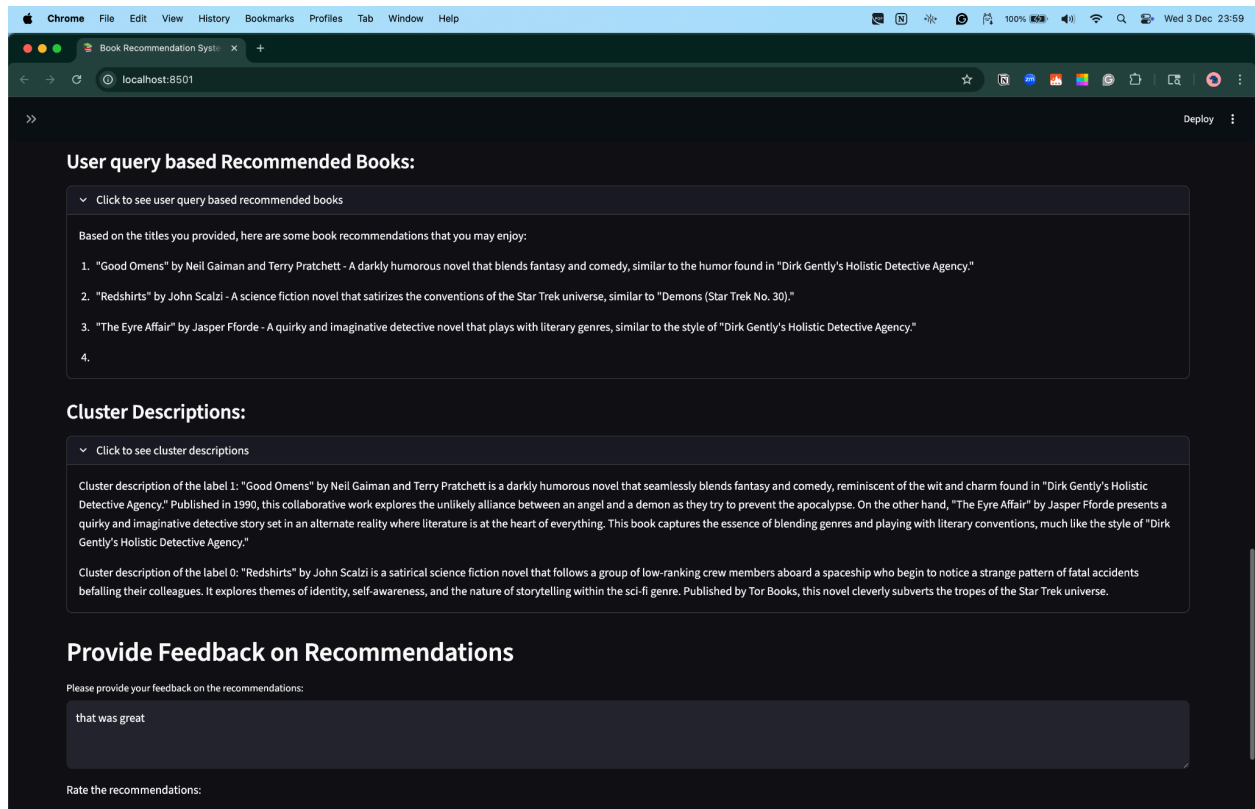


Image 3: Cluster descriptions and feedback front demo page

To guarantee the system's effectiveness for real-world use cases, the results can be evaluated based on rigorous metrics, including the Precision-Recall curve, RMSE, MAP, and relevance judgment scoring such as discounted cumulative gains. The implementation of the project uses a suite of advanced tools, including OpenAI, Streamlit, FastAPI, scikit-learn k-means algorithm, pandas, numpy, and Matplotlib to handle everything from user interaction to data visualisation. For a more detailed overview Dataset and GitHub code repository links are attached in the appendix section of the report.

Implementation approach

The generation of the cluster is performed using the K-Means algorithm, which clusters the data into two categories. Before the project was implemented using a robust Python-based technology stack, utilising Pandas for detailed data manipulation, Scikit-learn for machine learning tasks such as collaborative filtering, clustering and similarity metrics, and Streamlit for the frontend user interface of the system. Project utilises the helper functions, the reranker function, and OpenAI APIs to call the large language model for the various recommendation tasks. The project is developed on the local environment using git Git-based version control system that pushes the code to

the GitHub repository. It follows the conventions of Python programming and variable assignment rules for readability and reproducibility.

The system relies on a comprehensive Book Recommendation Dataset sourced from Kaggle, which initially comprised over 271,000 book titles and 800,000 user ratings. To manage computational resources effectively, this dataset was sampled down to approximately 100,000 titles. Data preparation involved rigorous cleaning and merging of three separate files—users, books, and ratings—to create a unified structure. The core recommendation engine is built upon an item-based collaborative filtering model that uses a user-item interaction matrix to calculate similarity scores between books. This is augmented by the integration of the OpenAI API, where the LLM interprets the raw outputs of the filtering model—often just book titles or IDs—to generate context-aware descriptions, rerank the book titles generated by the collaborative filtering, and identify additional, semantically related recommendations. Overall, the dataset had the following variables and features that were processed.

Variable Name	Source File	Data Type	Description
ISBN	Books.csv / Ratings.csv	String	International Standard Book Number.
Book-Title	Books.csv	String	The title of the book.
Book-Author	Books.csv	String	The name of the book's author
Year-Of-Publication	Books.csv	Integer/String	The year the book was published.
Publisher	Books.csv	String	The name of the publishing house.
Image-URL-S	Books.csv	String	URL pointing to a small-sized cover image of the book
Image-URL-M	Books.csv	String	URL pointing to a medium-sized

			cover image of the book.
Image-URL-L	Books.csv	String	URL pointing to a large-sized cover image of the book.
User-ID	Users.csv / Ratings.csv	Integer	An anonymised unique identifier for each user.
Location	Users.csv	String	The user's location,
Age	Users.csv	Float/Integer	The age of the user.
Book-Rating	Ratings.csv	Integer	The rating given by a user to a specific book.

Table 1: Data variable description

The project is structured through the various functions and classes so that it is efficient and seamless to implement in the production environment. For the detailed implementation demo and processed code, visit the link given in the appendix section of the report.

Repository structure

```

/book-recommendation-system
├── environment.yml          # Conda environment file for dependencies
├── .gitignore               # Git ignore file
├── LICENSE                  # Project license file
├── README.md                # This README file
├── /data/                   # Directory for storing datasets (e.g., ratings, books info)
├── /nbs/                    # Directory for Jupyter notebooks (e.g., EDA, model prototyping)
├── /feedback/               # user feedback folder
├── /assets/                 # artifacts folder
├── /tests/                  # code test file
├── ├── main.py              # FastAPI application entry point, defines API endpoints
├── ├── collabfiltering.py   # Module for collaborative filtering logic
├── ├── llmrec.py            # Module for LLM-based recommendation logic
├── ├── clustering.py        #
├── ├── helpers.py           # helper functions for the app.py
├── ├── Dockerfile           # docker file for the project packaging
├── └── app.py                # Main Streamlit application file (the user interface)

```

Image 4: Repository structure

System Diagram

Below is the system diagram of the project with a high-level design overview of each component and system functions, with a suitable application programming interface design.

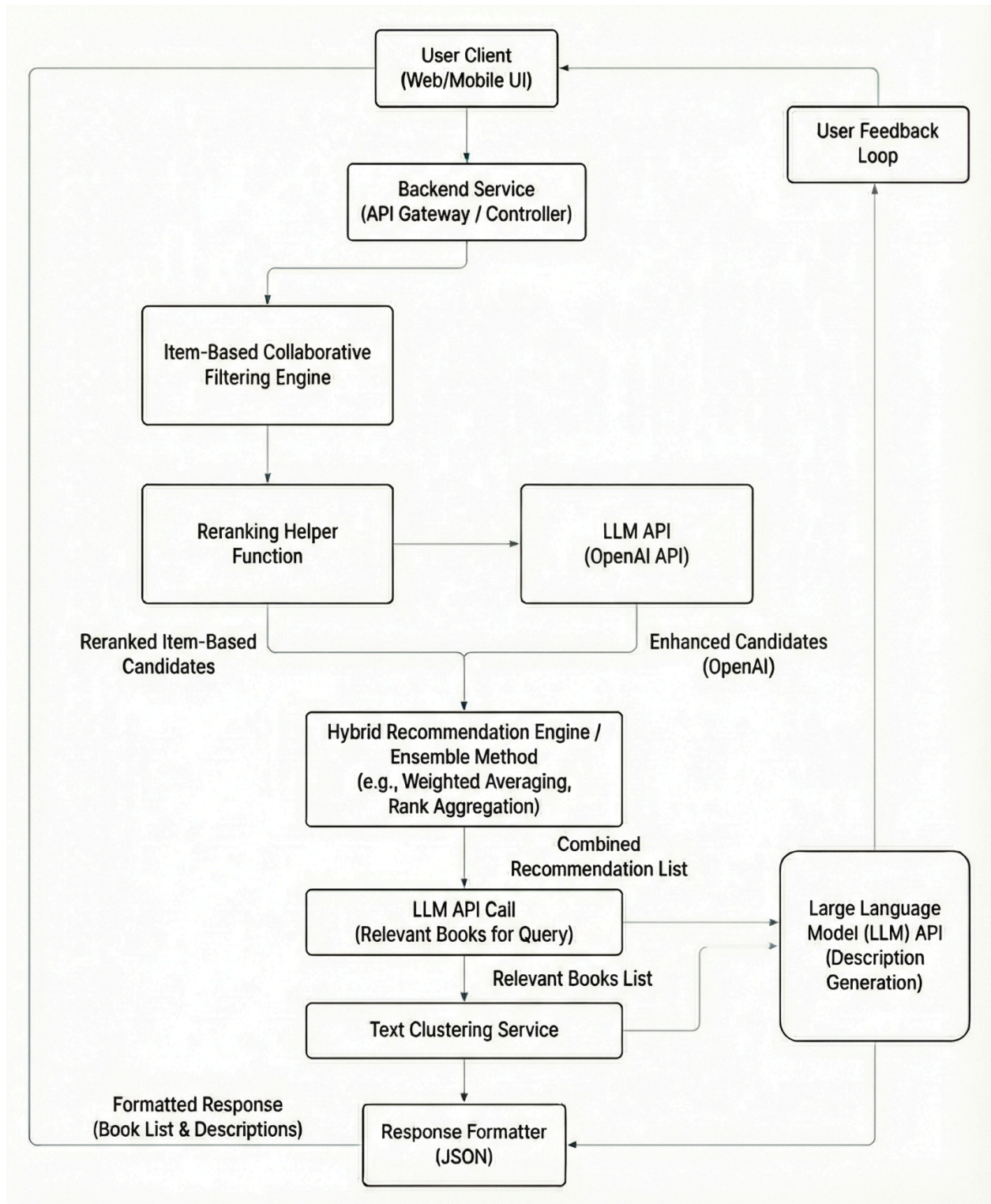


Figure 2: System design diagram of the book recommendation system

Challenges

Developing this hybrid system presented several significant technical challenges that required careful problem-solving and a thoughtful implementation approach. One of the primary hurdles was the sheer volume of the original dataset, which exceeded 100MB and contained over 800,000 rows. Processing this full dataset posed severe memory constraints, particularly when constructing the dense user-item matrix required for collaborative filtering. Pivoting the dataset is a very memory-intensive task, so it requires higher time complexity to perform calculations closely related to this. One of the issues with calculating cosine similarity scores were resource-intensive. To mitigate these performance issues without significantly sacrificing recommendation quality, the dataset was strategically sampled to a manageable size of 100,000 titles. Additionally, selecting the right mix of algorithms proved critical, as it balanced the strengths of collaborative filtering against its "cold start" weaknesses. Finally, the system had to bridge the gap between user behaviour and data structure; while the dataset relies on ISBNs as unique identifiers, users inevitably search by title. Developing a reliable mapper to translate user input into system-readable IDs was essential for the accurate functioning of the collaborative filtering model. Since the input of the recommendation engine would be an ISBN, the LLM call will have input of the outputs of the collaborative filtering, which are book titles.

Reflection

This project provided valuable insights into the complexities and potential of modern information retrieval systems. The development of a hybrid recommendation system highlighted the distinct advantages of combining different methodologies for a wide range of recommendation tasks. I learned that the synergy between collaborative filtering item-based similarity and Large Language Models significantly enhances retrieval capabilities; while collaborative filtering excels at identifying patterns in user behaviour, LLMs provide the necessary semantic nuance to understand the content of a book.

Furthermore, implementing clustering algorithms proved highly effective in organising the output and grouping the output to generate a description of each cluster, which enhances the user's understanding of the recommendations. Instead of presenting a flat, overwhelming list, grouping books by author or theme created a much more user-centric experience, allowing users to explore "pockets" of relevant content. The project also demonstrated the power of the LLM as a retrieval and reranker enhancer, capable of "reading" and describing vast amounts of embedded data to improve the

ranking process and provide context on *why* a specific book was recommended—a feature often missing in traditional matrix-factorisation models. LLM augmentation can have a really precise application to improve existing systems with alignment to user needs, which I learned from this project. Although it demonstrates the underlying process and functions of the system, the project still has room to develop and improve so that it is more user-friendly.

References

1. Project Code Repository:
<https://github.com/avikumart/Book-recommendation-system>
2. Course Material: INFO 556 Information Retrieval and Web Search, University of Arizona, By: Dr Xiao Hu, Assistant Professor, UofA
3. Data Source:
<https://www.kaggle.com/datasets/arashnic/book-recommendation-dataset/data>

Appendix

1. Book Recommendation Dataset Link (Source):
<https://www.kaggle.com/datasets/arashnic/book-recommendation-dataset/data>
2. GitHub Rep link: <https://github.com/avikumart/Book-recommendation-system>
3. Preprocessed data source:
<https://drive.google.com/drive/folders/1tIQ6RqewSVmPo-cqF1K3uD-h58Zekpl5?usp=sharing>
4. Project Demo link:
<https://drive.google.com/file/d/11YSVltXeSnKbpal942vA1t8BX3n0pj09/view?usp=sharing>
5. Streamlit app code:
<https://github.com/avikumart/Book-recommendation-system/blob/main/app.py>
6. Figure 1: Introductory image of the hybrid recommendation image
7. Image 1: Book recommendation system front demo page
8. Image 2: LLM-based and combined recommendation front demo page
9. Image 3: Cluster descriptions and feedback front demo page
10. Table 1: Data variable description
11. Image 4: Repository structure
12. Figure 2: System design diagram of the book recommendation system